INFORME PRELIMINAR – DESAFÍO 2

Amigos para siempre, con Ude@Book

Curso: Informática II

Semestre: 2026-2

Alejandro Macias Rodríguez

CONTEXTUALIZACION:

El desafío consiste en desarrollar un programa que simule las funcionalidades básicas de una red social denominada Ude@Book, utilizando el paradigma de programación orientada a objetos.

La plataforma debe permitir la administración de usuarios, amistades (solicitudes y amistades ya establecidas), álbumes de fotografías y mensajes. También debe ofrecer funcionalidades como el inicio de sesión, la visualización de fotografías de amigos (solo las rutas), las sugerencias de amistad y la consulta de mensajes.

Debemos también tener presente la eficiencia en el uso de los recursos computacionales, por lo que se requiere implementar estructuras de datos propias mediante memoria dinámica y medir tanto las iteraciones ejecutadas como el consumo de memoria de las funcionalidades.

Ya como consideraciones finales del contexto de este problema, debemos tener presente las prohibiciones de: herencia, struct y STL.

ANALISIS DEL PROBLEMA:

Es necesario identificar los elementos que conforman la plataforma, las relaciones existentes entre ellos y las operaciones que se deben realizar sobre la información, a continuación, explico lo entendido acerca de las funcionalidades:

Gestión de usuarios y almacenamiento: Cada usuario debe contar con un nombre de usuario único y sus respectivos datos personales. El sistema debe permitir recuperar la información desde archivos (almacenamiento permanente) y mantener actualizados los datos que sean modificados durante la ejecución (almacenamiento temporal durante el tiempo de ejecución). También debe garantizar que las relaciones entre usuarios, mensajes, álbumes y fotografías se conserven correctamente.

Relaciones de amistad: Las amistades deben ser mutuas, de manera que cuando dos usuarios establezcan una amistad, ambos queden relacionados. Además, se debe evitar el envío de solicitudes a usuarios que ya sean amigos o con quienes exista una solicitud pendiente que impida realizar la operación.

Visualización de amigos: Al iniciar sesión, se deben seleccionar aleatoriamente tres fotografías de perfil de amigos para ser mostradas. Se debe evitar repetir fotografías dentro de una misma selección y controlar que la actualización del grupo de fotografías solamente pueda realizarse una vez.

Administración de álbumes y fotografías: Cada usuario puede poseer varios álbumes con diferentes niveles de privacidad. Por lo tanto, se debe controlar cuáles álbumes pueden ser

consultados por otros usuarios. Las fotografías deben poder recorrerse circularmente y permitir aumentar o disminuir su cantidad de likes mediante las operaciones correspondientes.

Sugerencias de amistad: El sistema debe identificar usuarios que sean amigos de los amigos del usuario actual, sin incluir sus amistades existentes. Posteriormente, debe calcular la cantidad de amigos en común y ordenar los candidatos para presentar las sugerencias solicitadas. Para esto tenemos la sugerencia de la matriz de adyacencia, sin embargo, tenemos la libertad de decidir.

Gestión de mensajes: Los usuarios deben poder intercambiar mensajes con sus amigos, consultar los mensajes recibidos que todavía no han sido leídos y realizar búsquedas dentro de un intervalo de fechas. Para ello, se debe conservar la información del emisor, destinatario, contenido, fecha de envío y estado de lectura.

Medición de recursos: Cada funcionalidad debe permitir conocer la cantidad de iteraciones necesarias para ejecutarse, incluyendo las realizadas indirectamente. También se debe contabilizar la memoria utilizada por las estructuras de datos, objetos y variables.

DISEÑO:

Tengamos presente que somos nosotros mismos los que vamos a decidir acerca de como vamos a manejar los archivos, es decir, la base de datos de nuestra red social, por lo tanto, mi propuesta para la base de datos es a partir de los siguientes 6 archivos:

usuarios.txt: información de cada usuario (credenciales)

amistades.txt: registro de las relaciones de amistad.

solicitudes.txt: solicitudes de amistad pendientes.

mensajes.txt: mensajes enviados y recibidos.

albumes.txt: álbumes y niveles de privacidad.

fotos.txt: fotos, rutas y likes.

Ahora tenemos que plantear las clases que vamos a implementar:

Plataforma: coordinara todo el sistema, administrando usuarios, mensajes y solicitudes.

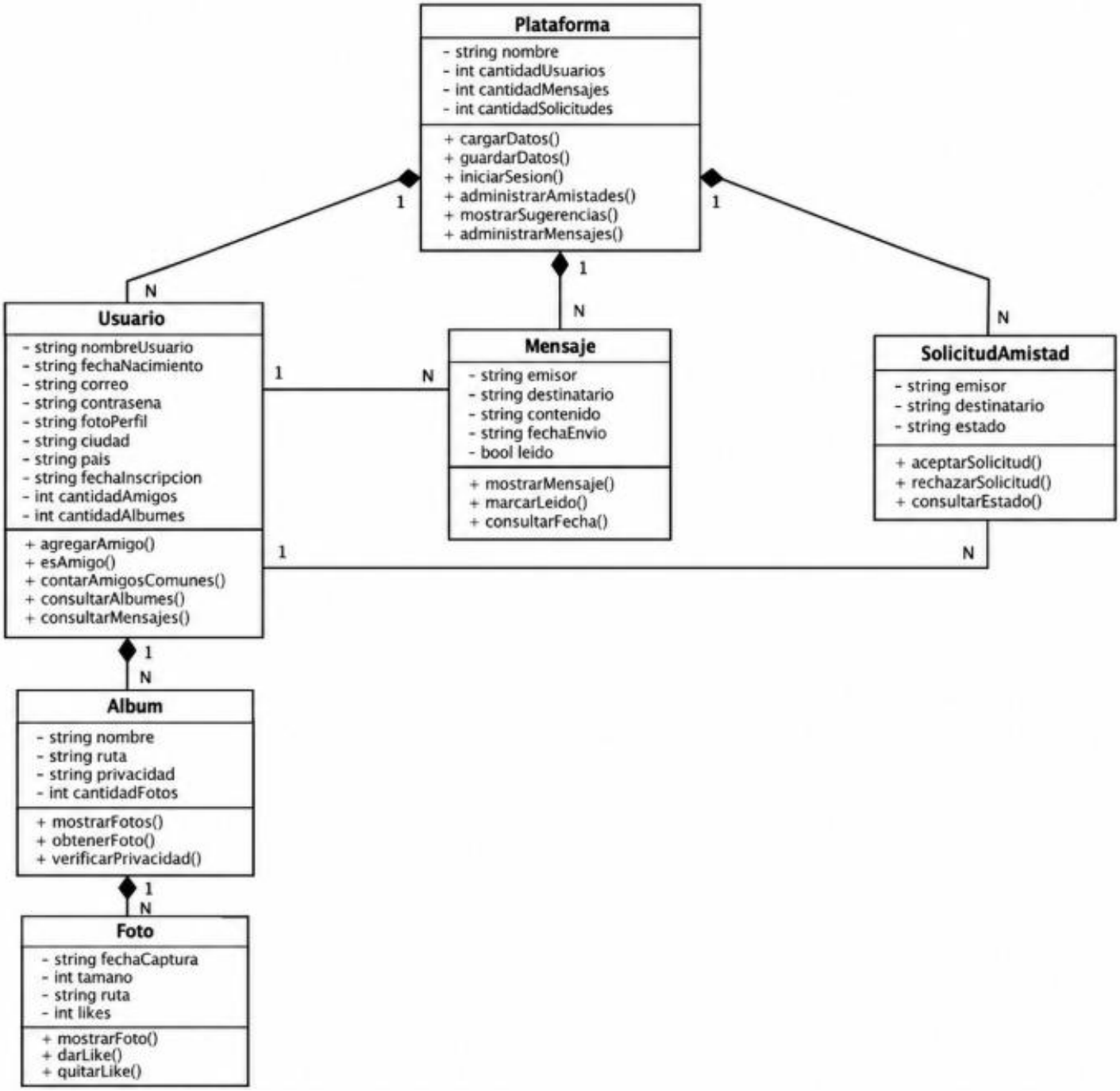
Usuario: representa una persona registrada, con sus datos, amigos y álbumes .

Álbum: agrupa fotos y establece quien puede visualizarlas.

Foto: representa una fotografía, con su ruta, fecha, tamaño y likes.

Mensaje: representa un mensaje entre dos usuarios.

SolicitudAmistad: Representa una solicitud enviada de un usuario a otro.



Finalmente dejo aquí anexado una foto de mi cuaderno donde tome un par de notas de análisis, rescato muchísimo una frase en especifico que escuche en una de las clases de socialización y creo que va a tener mucha relevancia en el desarrollo de este desafío:

“Es muy importante distinguir en todo momento lo almacenado de manera permanente de lo que esta almacenado en el programa durante la ejecución”

Esta semana de análisis he dado mucho con el pensamiento de que los objetos almacenados en la RAM durante la ejecución son los que permiten el funcionamiento de nuestra red social, pero los archivos son la memoria a largo plazo del desafío. Es como si en el anterior desafío nos hubieran pedido guardar el progreso de los usuarios, para que así siempre que el programa fuera ejecutado tuviera esa memoria de quien es cada quien y cuanto progreso dentro del juego tiene, aquí lo que debe pasar es que si yo estoy navegando por Ude@Book puedo dejar de ejecutar el programa y ejecutarlo en cualquier momento y aun así la aplicación siempre sabrá quienes son mis amigos, cuales son mis álbumes, que fotos tienen mis álbumes y en general toda la información de la red social.

Desafio 2.

nos piden desarrollar una especie de facebook en la consola que permita administrar: usuarios, amistades, fotografias, albumes y mensajes

Notas importantes:

- distinguir en todo momento lo almacenado de manera permanente de lo que esta almacenado en el programa durante la ejecucion
- diferencia entre clase y objeto: "molde de galletas" "una galleta en particular"
- Consultar sobre la matriz de adyacencia

